

ML project

Natália Chanová

January 2026

1 Introduction

Traffic Sign Recognition (TSR) is a fundamental component of modern Advanced Driver Assistance Systems (ADAS) and autonomous driving technologies. The ability of a vehicle to correctly identify commands, prohibitions, and warnings from visual inputs in real-time is essential for ensuring road safety.

The objective of this project is to design, implement, and thoroughly analyze a machine learning model capable of automatic traffic sign classification. In terms of problem definition, this is a supervised multi-class image classification task, where the goal is to learn a mapping function $f(x) = y$, which assigns the correct class label y from a predefined set of traffic sign types to an input image tensor x .

Although modern deep neural networks achieve high accuracy in this task, they face significant challenges in real-world deployment that are not always present in standard training datasets. These challenges include adverse lighting conditions, camera sensor noise, motion blur, or targeted adversarial attacks.

Therefore, this project is not limited to merely achieving high accuracy on a test set. The primary contribution of this work is a deep analysis of model robustness. Beyond training a Convolutional Neural Network (CNN) on the GTSRB dataset, this project investigates the model's behavior under simulated real-world conditions (darkness, noise, blur) and tests its resilience against the Fast Gradient Sign Method (FGSM) for generating adversarial examples. The work also attempts to address model explainability using the Grad-CAM technique, visualizing which parts of the image influence the network's decision.

2 Methodology

This chapter describes the dataset used, the data preprocessing steps, the architecture of the proposed neural network, and the training process. The project was implemented in Python using the PyTorch framework.

2.1 Dataset and Preprocessing

For training and evaluation purposes, the standardized GTSRB (German Traffic Sign Recognition Benchmark) dataset was used. The dataset contains over 50,000 images distributed into 43 classes (e.g., speed limits, stop signs, yield signs).

The data was split into a training and validation set with a ratio of 80:20. To ensure the model processes inputs uniformly, the following preprocessing steps were applied:

- **Resizing:** All images were downsampled to a fixed resolution of 32×32 pixels. This resolution represents a trade-off between preserving essential features (e.g., shape) and computational efficiency.
- **Normalization:** Pixel values (originally in the range $0 - 255$) were transformed into the interval $[-1, 1]$. This step is critical for accelerating the convergence of gradient descent during training.

2.2 Model Architecture (CNN)

The proposed model, TrafficSignNet, is a Convolutional Neural Network (CNN). The architecture consists of two main blocks:

2.2.1 Feature Extraction

- **Convolutional Layers:** The first layer uses 16 filters with a 5×5 kernel to capture coarse features (edges, colors). The second layer uses 32 filters with a 3×3 kernel to detect more complex shapes.
- **Activation Function (ReLU):** Introduces non-linearity into the network, allowing the model to learn complex relationships.
- **Max Pooling:** Layers with a 2×2 kernel reduce the spatial dimension of feature maps by half, thereby reducing the number of parameters and increasing invariance to small shifts in the image.

2.2.2 Classification Head

- The output from the convolutional part is flattened into a vector.
- This is followed by a Fully Connected layer with 120 neurons.
- **Dropout (0.5):** During training, 50% of neurons in this layer are randomly deactivated. This regularization technique effectively prevents overfitting by forcing the network to learn more robust features.
- The output layer consists of 43 neurons corresponding to the number of classes.

2.3 Training and Optimization

The training process was conducted over 10 epochs with a batch size of 64.

- **Loss Function:** Cross-Entropy Loss was used, which is the standard for multi-class classification tasks as it penalizes confident but incorrect predictions.
- **Optimizer:** The model was optimized using the Adam algorithm with a learning rate of 0.001. Adam combines the advantages of AdaGrad and RMSProp methods, ensuring fast convergence.

2.4 Experimental Setup

Three sets of experiments were designed beyond standard validation:

1. **Error Analysis:** Creation of a Confusion Matrix to identify classes that the model most frequently confuses.
2. **Robustness Test:** Evaluation of the model on a validation set modified with synthetic corruptions (Gaussian noise, Gaussian blur, brightness changes) to simulate real-world conditions.
3. **Adversarial Robustness:** Testing resilience against the FGSM (Fast Gradient Sign Method) attack, which adds invisible noise calculated from the loss gradient to the image with the intent of fooling the model.

3 Baseline Results

The training process was stable and converged rapidly. As shown in the training logs, the model achieved a validation accuracy of 93.94% as early as the second epoch. By the final (10th) epoch, the model reached a validation accuracy of **98.89%** with a validation loss of 0.0467.

The learning curves (Figure 1) show a consistent decrease in both training and validation loss, indicating that the model did not suffer from significant overfitting despite the relatively simple architecture.

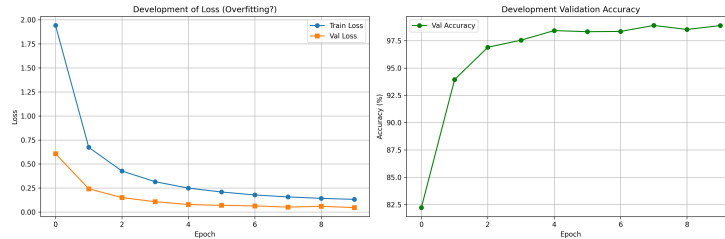


Figure 1: Training and validation loss and accuracy over 10 epochs.

3.1 Error Analysis

To better understand the model’s limitations, we analyzed the Confusion Matrix. The overall accuracy is high, but specific pairs of traffic signs remain challenging. The most frequent misclassifications occurred between signs with similar geometric shapes or similar internal layouts.

Table 1 highlights the most common errors found in the validation set:

True Class	Predicted Class	Count
Priority road (12)	Speed limit 30 (1)	9
Speed limit 30 (1)	Priority road (12)	6
Priority road (12)	Keep right (38)	5
Keep right (38)	Slippery road (23)	5
Slippery road (23)	Keep right (38)	4

Table 1: Top 5 most frequent misclassifications.

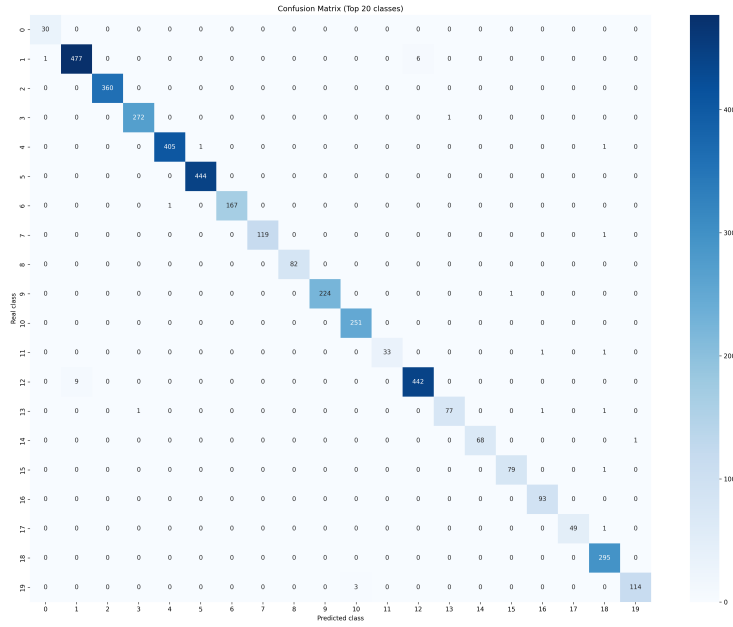


Figure 2: Confusion Matrix for the top 20 classes.

4 Robustness Analysis (Real-World Conditions)

In real-world driving scenarios, images are rarely perfect. We evaluated the trained model against various synthetic corruptions to simulate adverse conditions such as camera noise, blur, and low lighting.

The results, summarized in Table 2, reveal a dichotomy in the model’s performance:

- **Lighting and Blur:** The model is remarkably robust to lighting changes. Even with brightness reduced by 50%, the accuracy dropped by only 1.42%. Similarly, the model handled light and heavy blur well.
- **Noise Sensitivity:** The model showed a critical weakness to Gaussian noise. A noise level of $\sigma = 0.2$ caused a massive performance drop of 22.10%, reducing accuracy to 76.79%. This suggests the model relies heavily on high-frequency texture details that are easily destroyed by grain.

Condition	Accuracy (%)	Drop (%)
Baseline (Clean)	98.89	-
Light Blur	98.69	-0.20
Heavy Blur	97.50	-1.39
Brightness -50%	97.48	-1.42
Low Contrast	98.74	-0.15
Gaussian Noise ($\sigma = 0.1$)	90.27	-8.62
Gaussian Noise ($\sigma = 0.2$)	76.79	-22.10

Table 2: Model performance under simulated real-world corruptions.

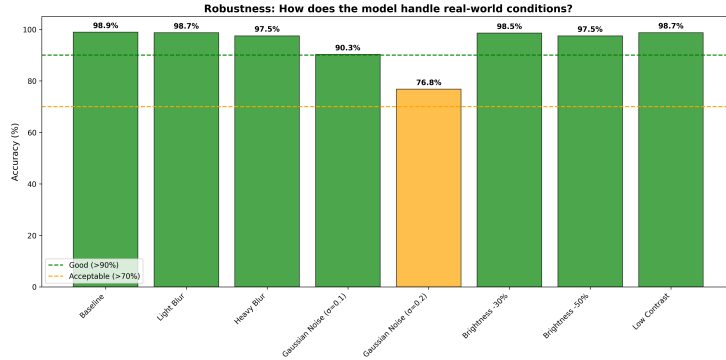


Figure 3: Impact of image corruptions on classification accuracy.

5 Adversarial Robustness

Finally, we tested the model’s security against the Fast Gradient Sign Method (FGSM) attack. This attack adds a small, calculated perturbation to the input image to maximize the loss function.

As hypothesized for standard CNNs without adversarial training, the model proved to be highly vulnerable.

- At a barely visible perturbation of $\epsilon = 0.01$, accuracy dropped instantly to 86.23%.
- At $\epsilon = 0.05$, the model was effectively broken, with accuracy falling to 41.48%.
- Stronger attacks ($\epsilon \geq 0.15$) reduced the model’s performance to that of random guessing ($< 20\%$).

This experiment confirms that while the model is accurate on standard data, it lacks the decision boundary margin required to resist targeted attacks.

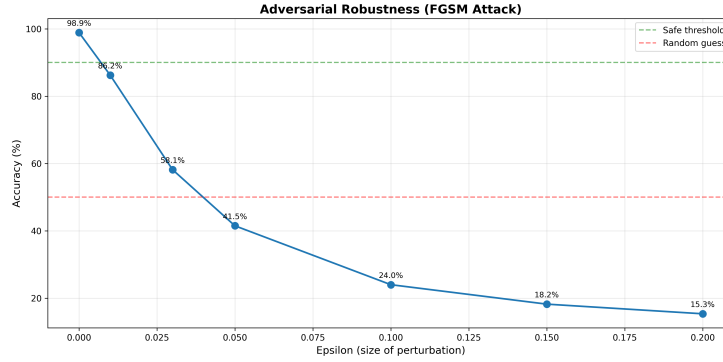


Figure 4: Degradation of accuracy under increasing intensity (ϵ) of FGSM attacks.

6 Model Interpretability

To ensure that the high accuracy of the model is not a result of learning irrelevant background features (e.g., sky color or trees), we employed the Grad-CAM (Gradient-weighted Class Activation Mapping) technique. This method visualizes the regions of the input image that were most important for the network’s prediction.

As illustrated in Figure 5, the results are mixed:

- **Correct Focus:** In most samples, the high activation areas (red regions) correspond to the geometric shapes and internal pictograms of the signs, confirming that the model has learned relevant features.
- **Background Bias:** However, in some instances (e.g., the 5th sample in our visualization), the activation map extends beyond the traffic sign boundary or focuses on background elements.

This observation suggests that for certain classes, the model might be relying on contextual cues (such as the road edge or background vegetation) rather

than the sign itself. This is a known phenomenon in deep learning known as "Clever Hans" behavior, which poses a risk if the model is deployed in a different environment where these contextual cues are absent.



Figure 5: Grad-CAM visualization. While the model generally focuses on the signs, some samples show activations spilling into the background, indicating potential context bias.

7 Conclusion

In this project, we successfully designed and implemented a Convolutional Neural Network for traffic sign classification using the GTSRB dataset. The baseline model achieved an excellent validation accuracy of **98.89%**, demonstrating that a custom, lightweight CNN architecture is sufficient for this task under standard conditions.

However, our analysis revealed critical insights into the model’s reliability in real-world scenarios:

1. **Strengths:** The model is highly robust to lighting changes and blur. It maintained over 97% accuracy even when image brightness was reduced by 50%, making it suitable for varying time-of-day conditions.
2. **Weaknesses:** The model is sensitive to sensor noise. A Gaussian noise level of $\sigma = 0.2$ caused a performance drop of over 22%.
3. **Vulnerability:** The adversarial test (FGSM) exposed a significant security risk. Small, targeted perturbations ($\epsilon = 0.05$) reduced the accuracy to roughly 41%, rendering the model unreliable under attack.

This project highlights that while achieving high accuracy is the first step, ensuring robustness and security is equally vital for safety-critical applications like autonomous driving.

The complete source code is available in the public GitHub repository:

<https://github.com/NataliaChanova/project-ML>